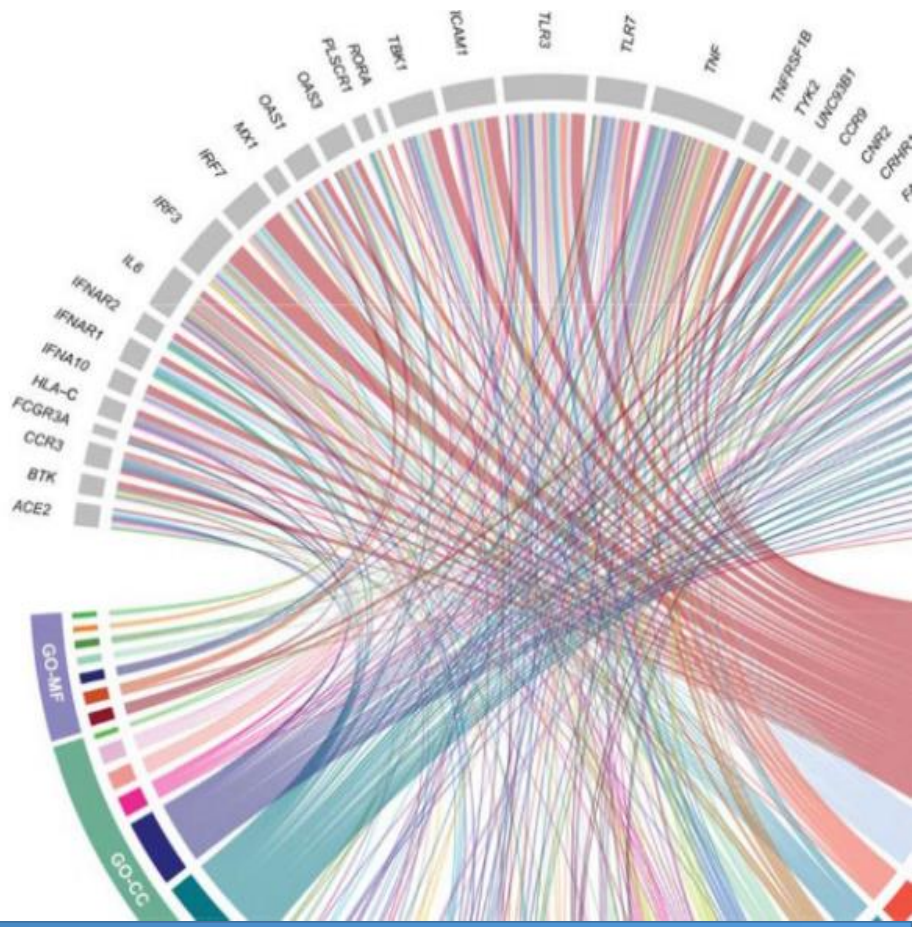
Tècniques i Eines Bioinformàtiques

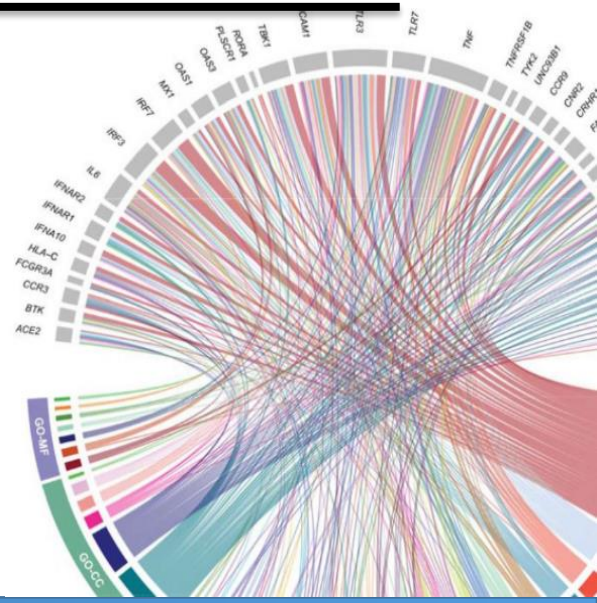
Wavefront Alignment Algorithm

Santiago Marco-Sola (santiago.marco@upc.edu)

Màster en Enginyeria Informàtica, UPC
Departament of Computer Science
Facultat d'Informàtica de Barcelona (FIB), UPC



1 Wavefront Alignment Algorithm (WFA)



2. Wavefront algorithm. Motivation

- Dynamic programming solutions are $O(n^2)$ in time and space
- Optimizations based on Dynamic programming do not scale
  - Banded: Needs to know maximum error-rate in advance.
  - SIMD: Complicated dependencies make vectorization difficult.

		G	A	A	T	A
	0	1	2	3	4	5
G	1	0	1	2	3	4
A	2	1	0	1	2	3
T	3	2	1	1	1	2
T	4	3	2	2	1	2
A	5	4	3	2	2	1
C	6	5	4	3	3	2
A	7	6	5	4	4	3

2. Wavefront algorithm. Idea (I)

- Diagonals are monotonically increasing.
  - For a given cell, if the characters of the text are matching, there is no better outcome as non increasing the value along the diagonal.
  - No need to know (or compute) the vertical/horizontal cells.
- **Diagonal Extension.**
  - Extending the furthest reaching cells on each diagonal until no match is found.
  - Each diagonal is independent from each other.
  - Character comparisons can be done in chunks.

		G	A	A	T	A
	0	1	2	3	4	5
G	1	0	1	2	3	4
A	2	1	0	1	2	3
T	3	2	1	1	1	2
T	4	3	2	2	1	2
A	5	4	3	2	2	1
C	6	5	4	3	3	2
A	7	6	5	4	4	3

2. Wavefront algorithm. Idea (I)

- Diagonals are monotonically increasing.
  - For a given cell, if the characters of the text are matching, there is no better outcome as non increasing the value along the diagonal.
  - No need to know (or compute) the vertical/horizontal cells.
- **Diagonal Extension.**
  - Extending the furthest reaching cells on each diagonal until no match is found.
  - Each diagonal is independent from each other.
  - Character comparisons can be done in chunks.

		G	A	A	T	A
	0	1	2	3	4	5
G	1					
A	2					
T	3					
T	4					
A	5					
C	6					
A	7					

2. Wavefront algorithm. Idea (I)

- Diagonals are monotonically increasing.
  - For a given cell, if the characters of the text are matching, there is no better outcome as non increasing the value along the diagonal.
  - No need to know (or compute) the vertical/horizontal cells.
- **Diagonal Extension.**
  - Extending the furthest reaching cells on each diagonal until no match is found.
  - Each diagonal is independent from each other.
  - Character comparisons can be done in chunks.

		G	A	A	T	A
	0	1	2	3	4	5
G	1	0				
A	2		0			
T	3					
T	4					
A	5					
C	6					
A	7					

2. Wavefront algorithm. Idea (II)

- Vertical and horizontal dependencies increment the value +1.
- The outcome cannot increment more than +1.
  - In the worst-case, if characters at that position don't match, the outcome will be incremented +1.
- We don't need to know all the surrounding cells.
  - We don't need to compute all cells to obtain the optimal alignment.
- From the **furthest reaching diagonal** (current, upper, or lower), we can compute the next-score cells.

		G	A	A	T	A
	0	1	2	3	4	5
G	1	0	1			
A	2	1	0	1		
T	3		1	1		
T	4					
A	5					
C	6					
A	7					

2. Wavefront algorithm. Idea (II)

- Vertical and horizontal dependencies increment the value +1.
- The outcome cannot increment more than +1.
  - In the worst-case, if characters at that position don't match, the outcome will be incremented +1.
- We don't need to know all the surrounding cells.
  - We don't need to compute all cells to obtain the optimal alignment.
- From the **furthest reaching diagonal** (current, upper, or lower), we can compute the next-score cells.
  - And extend again…

		G	A	A	T	A
	0	1	2	3	4	5
G	1	0	1			
A	2	1	0	1		
T	3		1	1	1	
T	4				1	
A	5					1
C	6					
A	7					

2. Wavefront algorithm. Idea (II)

- Vertical and horizontal dependencies increment the value +1.
- The outcome cannot increment more than +1.
  - In the worst-case, if characters at that position don't match, the outcome will be incremented +1.
- We don't need to know all the surrounding cells.
  - We don't need to compute all cells to obtain the optimal alignment.
- From the **furthest reaching diagonal** (current, upper, or lower), we can compute the next-score cells.
  - And extend again…

		G	A	A	T	A
	0	1	2	3	4	5
G	1	0	1			
A	2	1	0	1		
T	3		1	1	1	2
T	4		2		1	2
A	5					1
C	6					2
A	7					

2. Wavefront algorithm. Idea (II)

- Vertical and horizontal dependencies increment the value +1.
- The outcome cannot increment more than +1.
  - In the worst-case, if characters at that position don't match, the outcome will be incremented +1.
- We don't need to know all the surrounding cells.
  - We don't need to compute all cells to obtain the optimal alignment.
- From the **furthest reaching diagonal** (current, upper, or lower), we can compute the next-score cells.
  - And extend again…

		G	A	A	T	A
	0	1	2	3	4	5
G	1	0	1	?		
A	2	1	0	1	?	
T	3	?	1	1	1	2
T	4		2	?	1	2
A	5				?	1
C	6					2
A	7					

2. Wavefront algorithm. Idea (II)

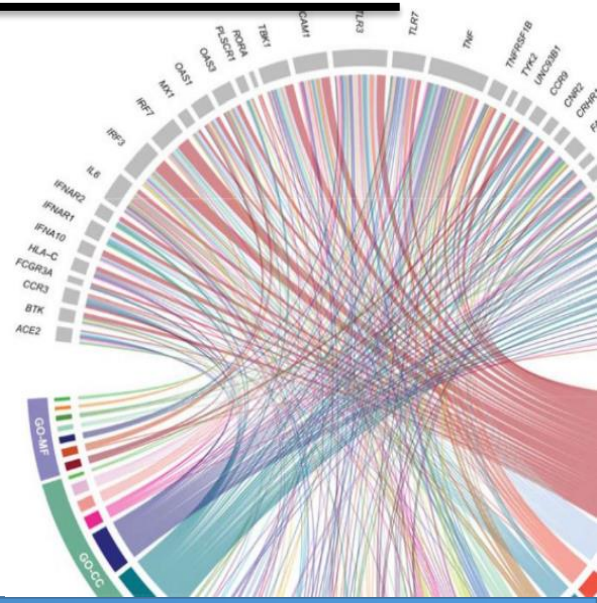
- Vertical and horizontal dependencies increment the value +1.
- The outcome cannot increment more than +1.
  - In the worst-case, if characters at that position don't match, the outcome will be incremented +1.
- We don't need to know all the surrounding cells.
  - We don't need to compute all cells to obtain the optimal alignment.
- From the **furthest reaching diagonal** (current, upper, or lower), we can compute the next-score cells.
  - And extend again…

		G	A	A	T	A
	0	1	2	3	4	5
G	1	0	1	?		
A	2	1	0	1	?	
T	3	?	1	1	1	2
T	4		2	?	1	2
A	5				?	1
C	6					2
A	7					

e=1 e=2

2

WFA Implementation



WFA. Rearrange the layout

(1) Align diagonals by shifting up columns (dependencies are kept homogeneous)

(2) Encode using diagonal offsets (instead of scores)

		G	A	A	T	A
	0	1	2	3	4	5
G	1	0	1	2	3	4
A	2	1	0	1	2	3
T	3	2	1	1	1	2
T	4	3	2	2	1	2
A	5	4	3	2	2	1
C	6	5	4	3	3	2
A	7	6	5	4	4	3

					A
				T	4
			A	3	3
		A	2	2	2
	G	1	1	1	2
G	0	0	1	1	1
A	1	1	2	2	2
T	2	2	2	3	3
T	3	3	3	4	
A	4	4	4		
C	5	5			
A	6				

				5
			5	6
		4	5	7
	2	5	6	7
		2	5	6
			3	5
				3

WFA. Computing wavefronts

Code

```
for score in {0 .. Optimal_score} do
  (1) Extend wavefronts matching chars
  (2) Compute the next wavefront
```

		0	1	2	3	4	5
k=+5							A
k=+4						T	4
k=+3					A	3	3
k=+2				A	2	2	2
k=+1			G	1	1	1	2
k = 0	G	0	0	1	1	1	1
k=-1	A	1	1	2	2	2	
k=-2	T	2	2	2	3	3	
k=-3	T	3	3	3	4		
k=-4	A	4	4	4			
k=-5	C	5	5				
k=-6	A	6					

	d=0	d=1	d=2	d=3
k=+3				
k=+2				
k=+1				
k = 0	0			
k=-1				
k=-2				
k=-3				

WFA. Computing wavefronts

Code

```
for score in {0 .. Optimal_score} do
  (1) Extend wavefronts matching chars
  (2) Compute the next wavefront
```

		0	1	2	3	4	5
k=+5							A
k=+4						T	4
k=+3					A	3	3
k=+2				A	2	2	2
k=+1			G	1	1	1	2
k = 0	G	0	0	1	1	1	
k=-1	A	1	1	2	2	2	
k=-2	T	2	2	2	3	3	
k=-3	T	3	3	3	4		
k=-4	A	4	4	4			
k=-5	C	5	5				
k=-6	A	6					

	d=0	d=1	d=2	d=3
k=+3				
k=+2				
k=+1				
k = 0	2			
k=-1				
k=-2				
k=-3				

WFA. Computing wavefronts

Code

```
for score in {0 .. Optimal_score} do
  (1) Extend wavefronts matching chars
  (2) Compute the next wavefront
```

		0	1	2	3	4	5
k=+5							A
k=+4						T	4
k=+3					A	3	3
k=+2				A	2	2	2
k=+1			G	1	1	1	2
k = 0	G	0	0	1	1	1	1
k=-1	A	1	1	2	2	2	
k=-2	T	2	2	2	3	3	
k=-3	T	3	3	3	4		
k=-4	A	4	4	4			
k=-5	C	5	5				
k=-6	A	6					

	d=0	d=1	d=2	d=3
k=+3				
k=+2				
k=+1		3		
k = 0	2	3		
k=-1		2		
k=-2				
k=-3				

WFA. Computing wavefronts

Code

```
for score in {0 .. Optimal_score} do
  (1) Extend wavefronts matching chars
  (2) Compute the next wavefront
```

		0	1	2	3	4	5
k=+5							A
k=+4						T	4
k=+3					A	3	3
k=+2				A	2	2	2
k=+1			G	1	1	1	2
k = 0	G	0	0	1	1	1	1
k=-1	A	1	1	2	2	2	
k=-2	T	2	2	2	3	3	
k=-3	T	3	3	3	4		
k=-4	A	4	4	4			
k=-5	C	5	5				
k=-6	A	6					

	d=0	d=1	d=2	d=3
k=+3				
k=+2				
k=+1		4		
k = 0	2	5		
k=-1		2		
k=-2				
k=-3				

WFA. Computing wavefronts

Code

```
for score in {0 .. Optimal_score} do
  (1) Extend wavefronts matching chars
  (2) Compute the next wavefront
```

		0	1	2	3	4	5
k=+5							A
k=+4						T	4
k=+3					A	3	3
k=+2				A	2	2	2
k=+1			G	1	1	1	2
k = 0	G		0	0	1	1	1
k=-1	A		1	1	2	2	2
k=-2	T		2	2	2	3	3
k=-3	T		3	3	3	4	
k=-4	A		4	4	4		
k=-5	C		5	5			
k=-6	A		6				

	d=0	d=1	d=2	d=3
k=+3				
k=+2			5	
k=+1		4	5	
k = 0	2	5	6	
k=-1		2	5	
k=-2			2	
k=-3				

WFA. Computing wavefronts

Code

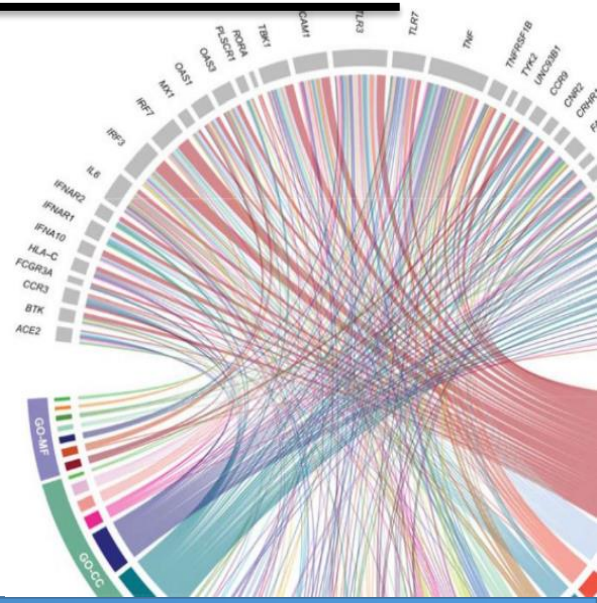
```
for score in {0 .. Optimal_score} do
  (1) Extend wavefronts matching chars
  (2) Compute the next wavefront
```

		0	1	2	3	4	5
k=+5							A
k=+4						T	4
k=+3					A	3	3
k=+2				A	2	2	2
k=+1			G	1	1	1	2
k = 0	G	0	0	1	1	1	1
k=-1	A	1	1	2	2	2	
k=-2	T	2	2	2	3	3	
k=-3	T	3	3	3	4		
k=-4	A	4	4	4			
k=-5	C	5	5				
k=-6	A	6					

	d=0	d=1	d=2	d=3
k=+3				5
k=+2			5	6
k=+1		4	5	7
k = 0	2	5	6	7
k=-1		2	5	6
k=-2			3	5
k=-3				3

3

Generalized WFA



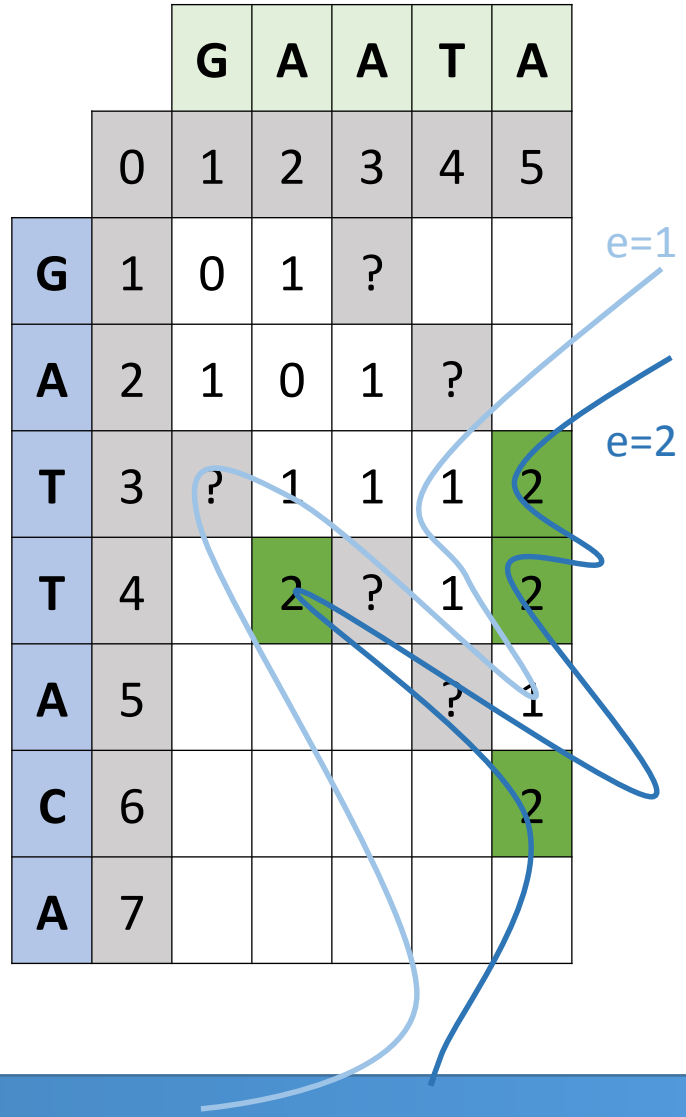
Accelerating Pairwise Alignment (WFA)

In sequence analysis, **pairwise alignment** compares two sequences in order to find similarities and differences.

> *"How to **convert one sequence into the other** applying a series of alignment operations that minimize a given cost function or distance function"*

Classical pairwise alignment implementations (software and hardware) are based on **dynamic programming algorithms** (i.e., Needleman-Wunsch, Smith Waterman).

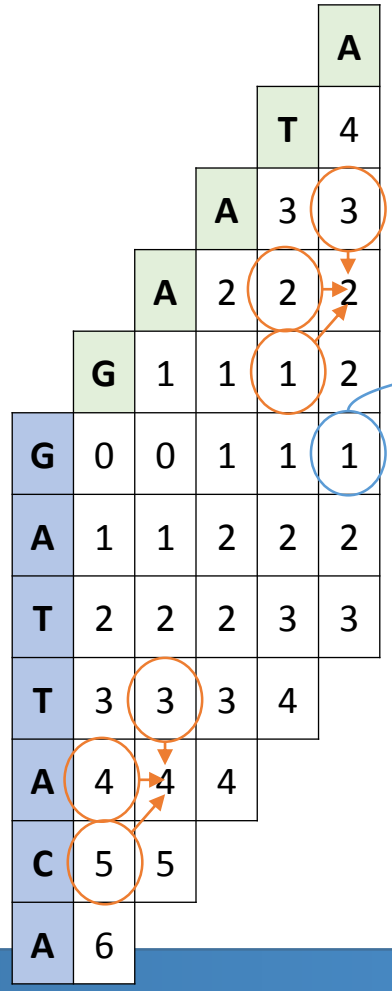
A novel approach, the **Wavefront Alignment algorithm (WFA)**, runs in **O(ne)**, avoiding the computation of multiple cells, using a simple computation pattern.

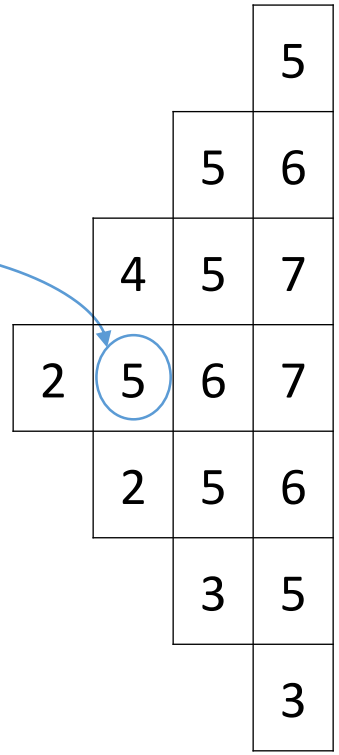


Accelerating Pairwise Alignment (WFA)

In a nutshell, the **WFA** method rearranges the layout to compute the **sequence alignment using fewer steps** and a simpler computation pattern

		G	A	A	T	A
	0	1	2	3	4	5
G	1	0	1	2	3	4
A	2	1	0	1	2	3
T	3	2	1	1	1	2
T	4	3	2	2	1	2
A	5	4	3	2	2	1
C	6	5	4	3	3	2
A	7	6	5	4	4	3





Accelerating Pairwise Alignment (WFA)

In a nutshell, the **WFA** method rearranges the layout to compute the **sequence alignment using fewer steps**, using a simple computation pattern

		T	C	T	A	T	A	C	T	G	C	G	C	G	T	T	T	G	G	A	G	A	A	A	T	A	A	A	A	T	A	G	T
	0	8	10	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48	50	52	54	56	58	60	62	64	66	68	70
T	8	0	8	10	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48	50	52	54	56	58	60	62	64	66	68
C	10	8	0	8	10	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48	50	52	54	56	58	60	62	64	66
T	12	10	8	0	8	10	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48	50	52	54	56	58	60	62	64
T	14	12	10	8	4	8	14	16	14	20	22	24	26	28	26	28	30	36	38	40	42	44	46	48	46	52	54	56	58	56	62	64	62
T	16	14	12	10	12	4	12	14	16	18	20	22	24	26	28	26	28	34	36	38	40	42	44	46	48	50	52	54	56	58	60	62	64
A	18	16	14	12	10	12	4	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48	50	52	54	56	58	60	62
C	20	18	16	14	16	14	12	4	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48	50	52	54	56	58	60
T	22	20	18	16	18	16	14	12	4	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48	50	52	54	56	58
C	24	22	20	18	20	18	16	14	12	8	12	18	16	22	24	26	28	30	32	34	36	38	40	42	44	46	48	50	52	54	56	58	60
G	26	24	22	20	22	20	18	16	14	12	12	12	20	16	24	26	28	28	30	34	34	38	40	42	44	46	48	50	52	54	56	56	60
C	28	26	24	22	24	22	20	18	16	18	12	16	12	20	20	24	26	28	30	32	34	36	38	40	42	44	46	48	50	52	54	56	58
G	30	28	26	24	26	24	22	20	18	16	20	12	20	12	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48	50	52	54	56
C	32	30	28	26	28	26	24	22	20	22	16	20	12	20	16	24	26	28	30	32	34	36	38	40	42	44	46	48	50	52	54	56	58
G	34	32	30	28	30	28	26	24	22	20	24	16	20	12	20	20	24	26	28	30	32	34	36	38	40	42	44	46	48	50	52	54	56
T	36	34	32	30	32	30	28	26	24	26	24	24	20	20	12	20	20	24	26	28	30	32	34	36	38	40	42	44	46	48	50	52	54
T	38	36	34	32	34	32	30	28	26	28	28	26	24	22	20	12	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48	50	52
G	40	38	36	34	36	34	32	30	28	26	30	28	26	24	22	20	16	20	22	28	26	32	34	36	38	40	42	44	46	48	50	48	54
G	42	40	38	36	38	36	34	32	30	28	30	30	28	26	24	22	24	16	20	26	28	30	32	34	36	38	40	42	44	46	48	50	52
A	44	42	40	38	36	38	36	34	32	34	32	32	30	28	26	24	26	24	20	20	28	28	30	32	36	36	38	40	42	46	46	50	52
G	46	44	42	40	42	40	38	36	34	32	36	32	32	30	28	26	28	26	24	24	20	28	30	32	34	36	38	40	42	44	46	46	50
A	48	46	44	42	40	42	40	38	36	38	36	36	34	32	30	28	30	28	30	24	28	20	28	30	32	34	36	38	40	42	44	46	48
A	50	48	46	44	42	44	42	40	38	40	40	38	36	34	32	30	32	30	32	30	28	28	20	28	30	32	34	36	38	40	42	44	46
A	52	50	48	46	44	46	44	42	40	42	42	40	38	36	34	32	34	32	34	32	32	28	28	20	28	30	32	34	36	38	40	42	44
T	54	52	50	48	50	44	46	44	42	44	44	42	40	38	36	34	32	34	36	36	34	32	30	28	20	28	30	32	34	36	38	40	42
A	56	54	52	50	48	50	44	46	44	46	46	44	42	40	38	36	38	36	38	36	36	34	32	30	28	20	28	30	32	34	36	38	40
C	58	56	54	52	54	52	50	44	46	48	46	46	44	42	40	38	40	38	40	40	38	36	34	32	30	28	24	32	34	36	38	40	42
A	60	58	56	54	52	54	52	50	48	50	50	48	46	44	42	40	42	40	42	40	40	38	36	34	32	30	28	24	32	34	36	38	40
A	62	60	58	56	54	56	54	52	50	52	52	50	48	46	44	42	44	42	44	42	42	40	38	36	34	32	30	28	24	32	34	36	38
T	64	62	60	58	60	54	56	54	52	54	54	52	50	48	46	44	42	44	46	46	44	42	40	38	36	34	36	34	32	24	32	34	36
A	66	64	62	60	58	60	54	56	54	56	56	54	52	50	48	46	48	46	48	46	46	44	42	40	38	36	34	36	34	32	24	32	34
G	68	66	64	62	64	62	60	58	56	54	58	56	54	52	50	48	50	48	46	50	46	46	44	42	40	38	40	38	36	34	32	24	32
T	70	68	66	64	66	64	62	60	58	60	58	58	56	54	52	50	48	50	52	50	50	48	46	44	42	40	42	40	38	36	34	32	24

		T	C	T	A	T	A	C	T	G	C	G	C	G	T	T	T	G	G	A	G	A	A	A	T	A	A	A	A	T	A	G	T
	0																																
T		0																															
C			0																														
T				0	8	10	12	14	16	18	20	22	24																				
T				8	4	8	16	18	14	22	24																						
T				10		4			18																								
A				12	10	18	4																										
C				14	18			4																									
T				16	20	18			4	12	14	16	18	20	22	24																	
C				18	22				12	8	12	18	16	22	24																		
G				20	24				14	12	12	12	22	16																			
C				22					16	18	12	16	12		20																		
G				24					18	16	22	12		12	20	22	24																
C									20	22	16		12		16																		
G									22	20		16		12		20																	
T									24		24		20		12		20																
T													24			12	20	22	24														
G																20	16	20	22														
G																22		16	20														
A																24		24	20	20													
G																		24	24	20													
A																			24		20												
A																						20											
A																							20										
T																								20									
A																									20								
C																										24							
A																											24						
A																												24					
T																													24				
A																														24			
G																															24		
T																																24	

Accelerating Pairwise Alignment (WFA)

The **WFA** algorithm iterates over **2 steps**:

1. Compute next wavefront

   Computes the next column of offsets

2. Extend wavefront

   Computes contiguos matching characters from both sequences starting from 2 given offsets.

	d=0	d=1	d=2	d=3
k=+3				5
k=+2			5	6
k=+1		4	5	7
k = 0	2	5	6	7
k=-1		2	5	6
k=-2			3	5
k=-3				3

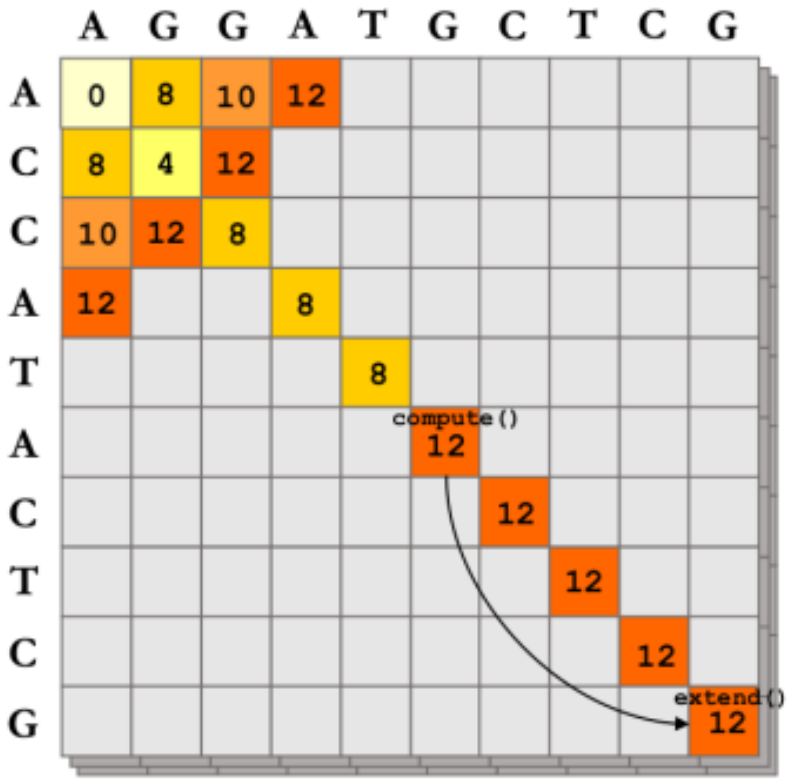
	d=0	d=1	d=2	d=3
k=+3				5
k=+2			5	6
k=+1		4	5	8
k = 0	2	5	6	9
k=-1		2	5	6
k=-2			3	6
k=-3				3

```
void edit_wavefronts_next_wavefront() {
  [...]
  // Compute next wavefront
  int k;
  for (k=lo+1;k<=hi-1;++k) {
    int max_ins_sub = MAX(offsets[k],offsets[k-1]) + 1;
    next_offsets[k] = MAX(max_ins_sub,offsets[k+1]);
  }
  [...]
}
```

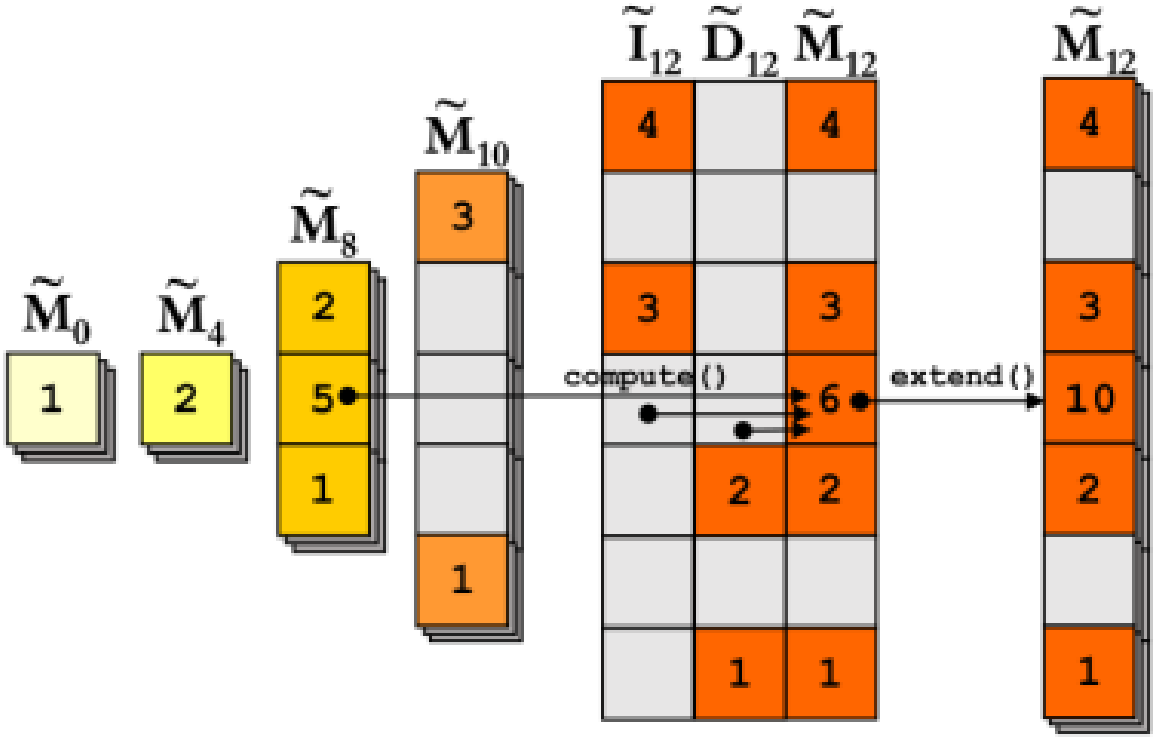
```
void edit_wavefronts_extend_wavefront() {
  [...]
  // Extend diagonally each wavefront point
  int k;
  for (k=k_min;k<=k_max;++k) {
    int v = offsets[k]-k, h = offsets[k], extended = 0;
    while (pattern[v++]==text[h++]) {
      ++extended;
    }
    offsets[k] += extended;
  }
}
```

WFA for gap-linear & gap-affine penalties

Dynamic Programming (DP)



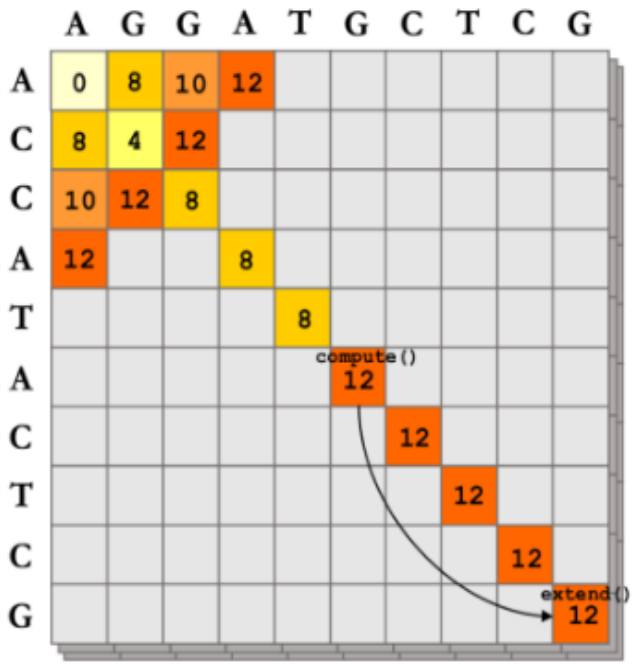
Wavefront Gap-Affine (WFA)



WFA for gap-linear & gap-affine penalties

- Adapt recurrences to fr-points on the diagonal layout
- WFA redefines classical DP equations based on the score and diagonal

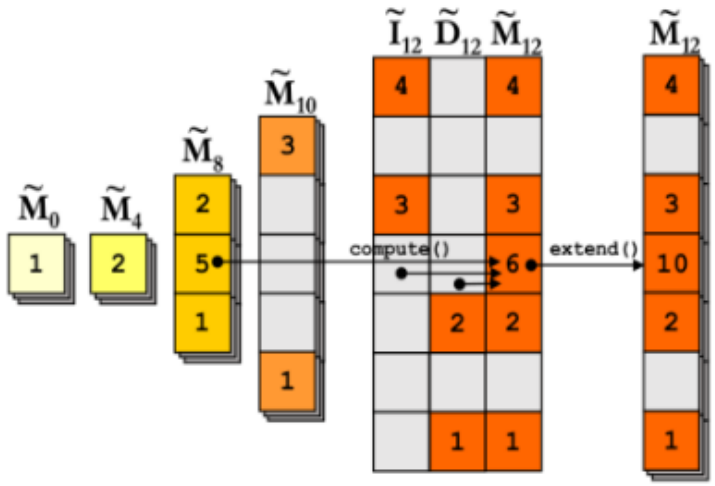
$$\begin{cases} I_{v,h} & = \min\{M_{v,h-1} + o + e, I_{v,h-1} + e\} \\ D_{v,h} & = \min\{M_{v-1,h} + o + e, D_{v-1,h} + e\} \\ M_{v,h} & = \min\{I_{v,h}, D_{v,h}, M_{v-1,h-1} + s(q_{v-1}, t_{h-1})\} \end{cases}$$



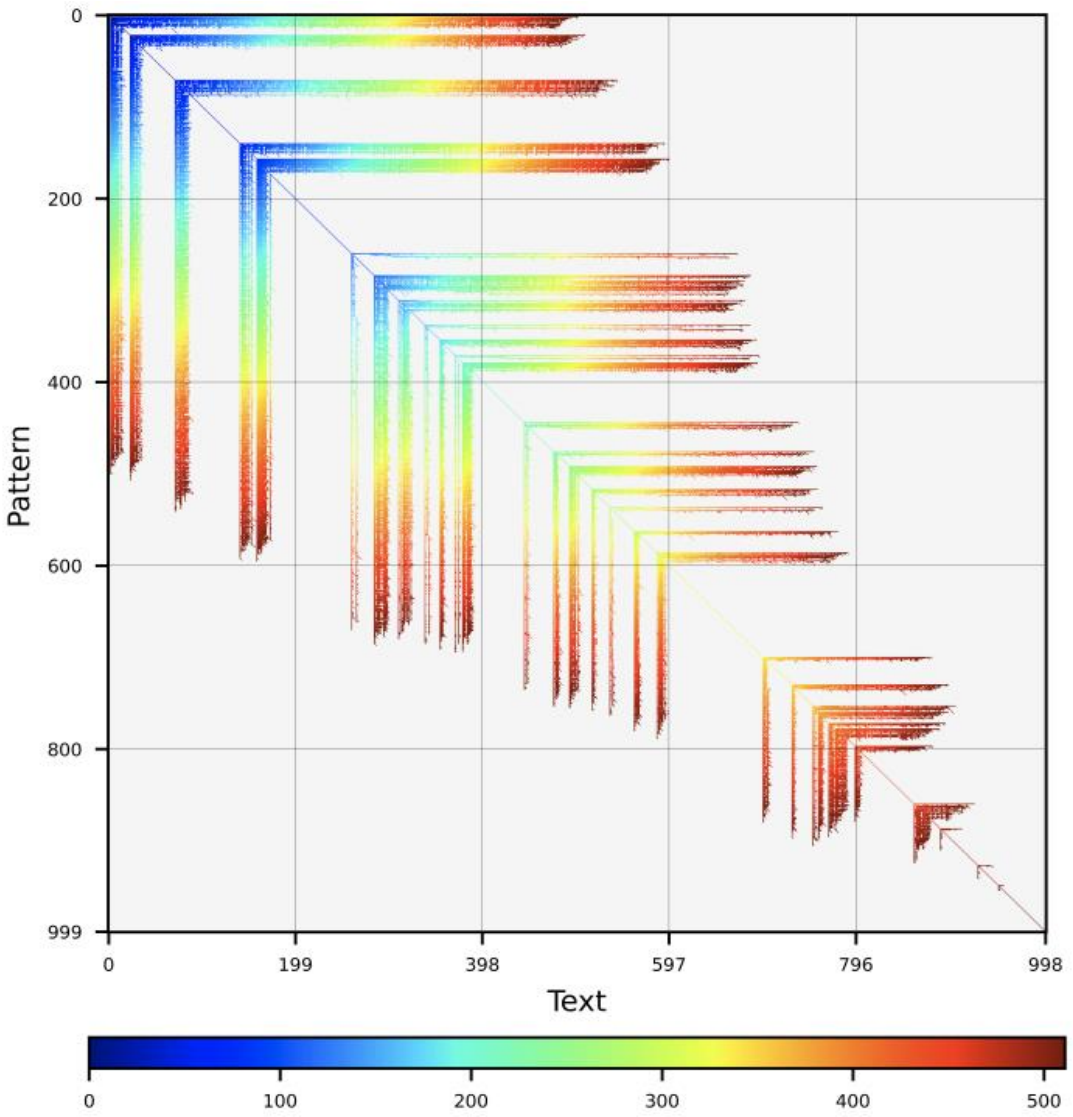
$$\tilde{I}_{s,k} = \max\begin{Bmatrix} \tilde{M}_{s-o-e,k-1} & \text{(Open insertion)} \\ \tilde{I}_{s-e,k-1} & \text{(Extend insertion)} \end{Bmatrix} + 1$$

$$\tilde{D}_{s,k} = \max\begin{Bmatrix} \tilde{M}_{s-o-e,k+1} & \text{(Open deletion)} \\ \tilde{D}_{s-e,k+1} & \text{(Extend deletion)} \end{Bmatrix}$$

$$\tilde{M}_{s,k} = \max\begin{Bmatrix} \tilde{M}_{s-x,k} + 1 & \text{(Substitution)} \\ \tilde{I}_{s,k} & \text{(Insertion)} \\ \tilde{D}_{s,k} & \text{(Deletion)} \end{Bmatrix},$$

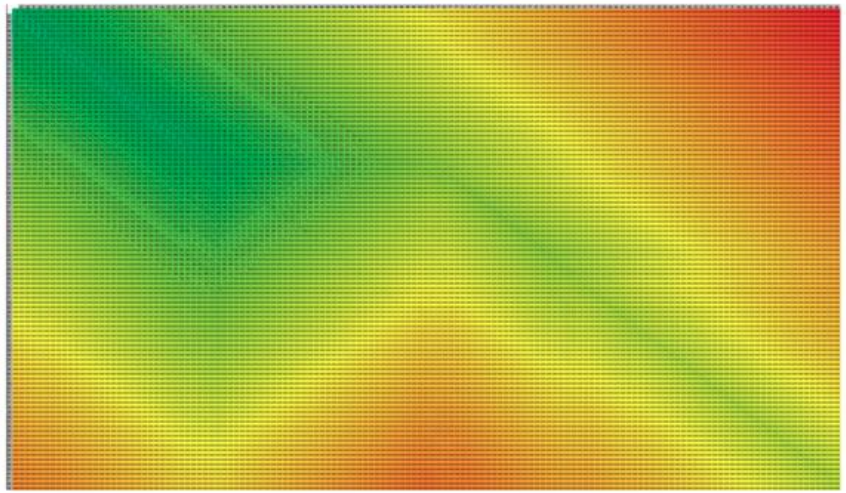


WFA equivalent DP-space exploration

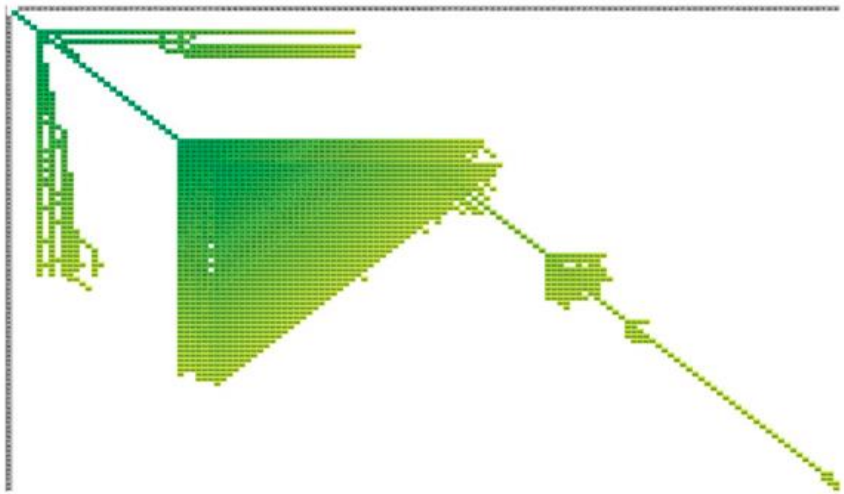


WFA Dynamic Reduction (further pruning)

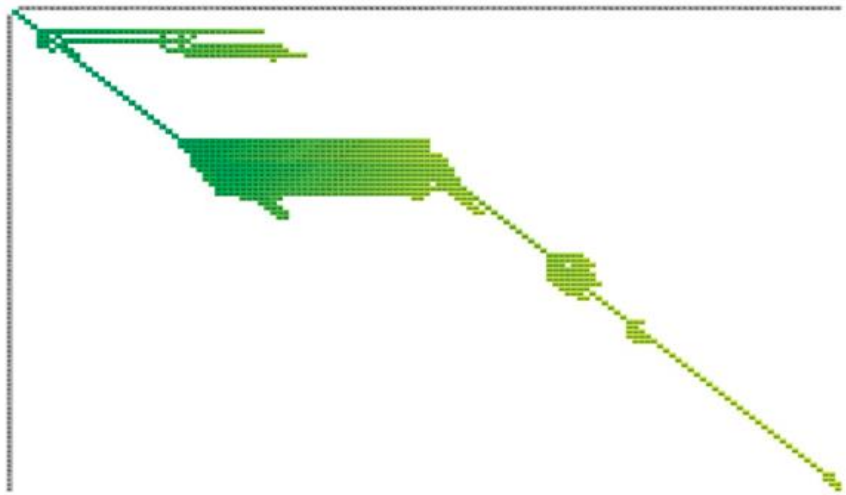
Dynamic Programming (DP)



Wavefront Gap-Affine (WFA)

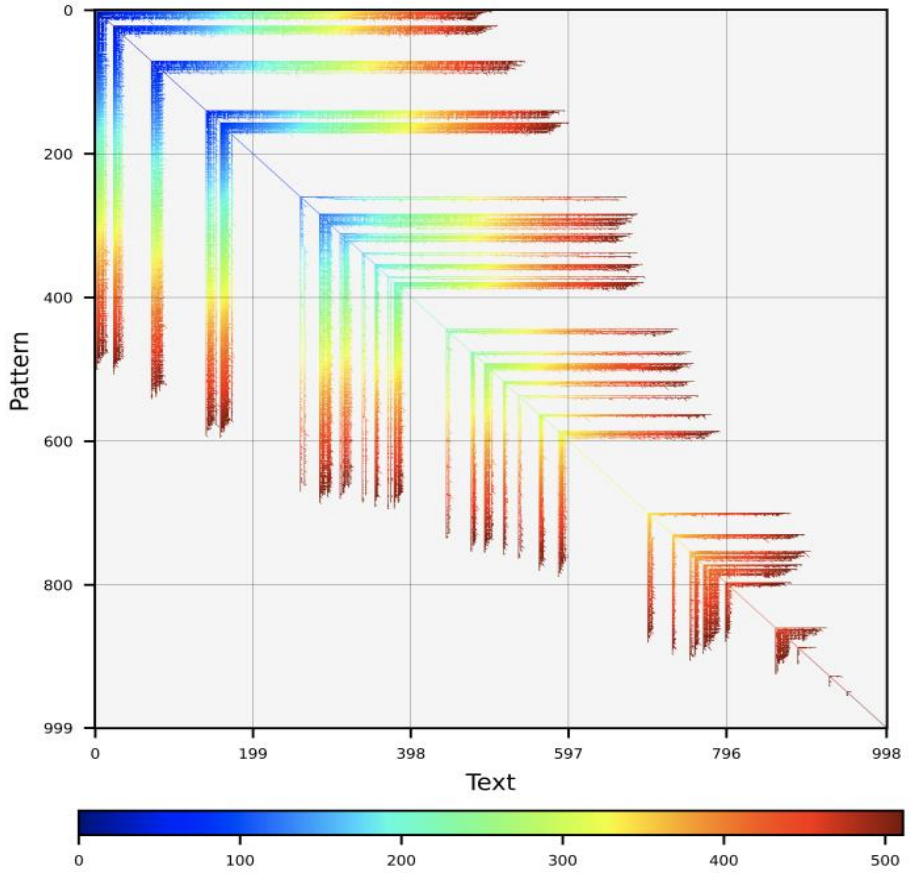


WFA Reduction

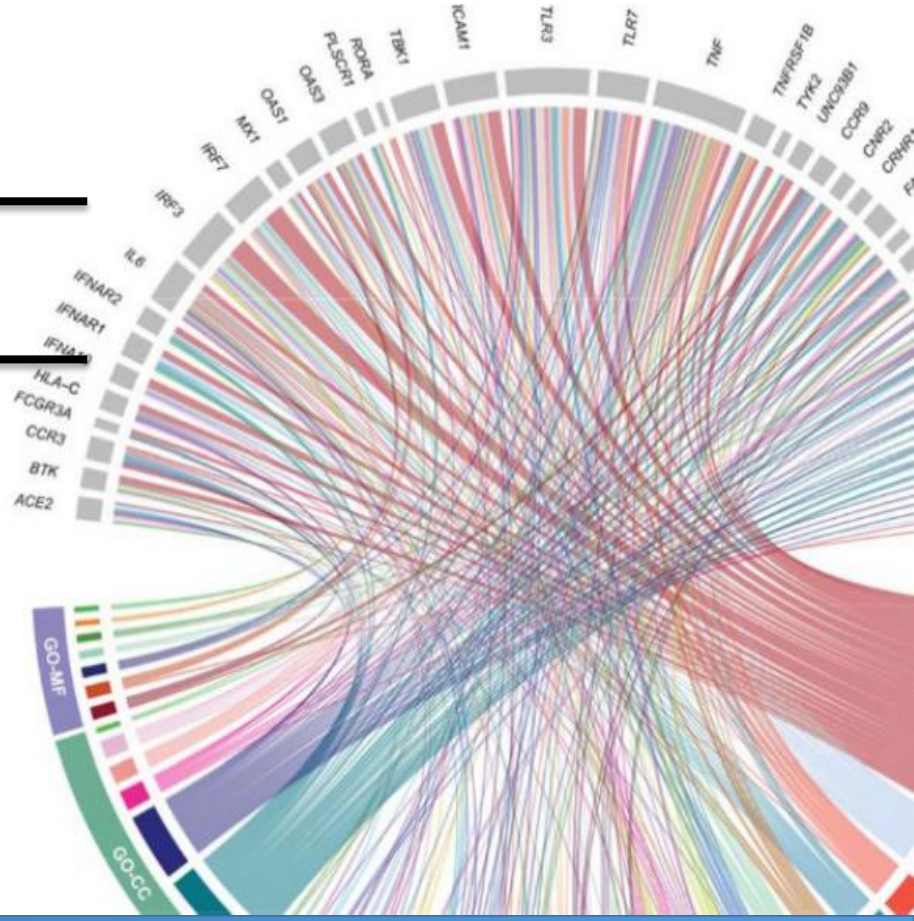


WFA in practical terms

- **Takes advantage of homologous regions** between the sequences, scaling with the read-length.
- Can be **automatically vectorized** using SIMD instructions for any CPU model.
- Because it encodes sequence position (and not scores) it can exploit 16-bits or 8-bits integers.
- **Alphabet independent** (uses ASCII encoded sequences).
- Support for different distance-functions.
  - Edit, gap-lineal, gap-affine, dual-cost gap-affine
  - Global and ends-free alignment support.
- Compatible with score-based drop/cut-off features.
- As a result, the WFA significantly outperforms other methods.
  - 20-300X faster than other methods aligning short Illumina-like sequences.
  - 10-100X faster using long noisy reads like those produced by Oxford Nanopore Technologies.



Questions


ACE2
BTK
CCR3
FCGR3A
HLA-C
IFNAR1
IFNAR2
IL6
IRF3
IRF7
MX1
OAS1
OAS3
PLSCR1
RORA
TBK1
ICAM1
TLR3
TLR7
TNF
TNFRSF1B
TYK2
UNC93B1
CCR9
CNR2
GO-MF
GO-CC